The AI I have produced for this coursework implements all of the requirements and passes all the test cases. I have cited the online resources that I used. In particular, Mark Karpov's Megaparsec tutorial [9] helped me get started.

1 Codebase Structure

`TUI.hs` - Provides a REPL for running the AI
`Grammar.hs` - Core building blocks for constructing parsers
`Numbers.hs` - Parsing and displaying numbers in numeric and longhand form
`Memory.hs` - Data type representing the AI's internal memory, and provides state actions to interact with it
`Error.hs` - Handles parser errors and tries to offer help where possible
`Helpers.hs` - Helper functions for setting up and printing to the terminal
`Skills.hs` - Aggregates skills and generates a full parser for them, along with a help command
`Skills/*.hs` - Collection of independent skills, detailed explanation given later

All of the code is well commented, explaining any nuances or justifying architectural decisions wherever applicable.

2 Monad Transformer Stack

The `Parser` type is given below. It layers `ParsecT` over `StateT` over `IO`. The inclusion `IO` allows complex parser behaviour, like fetching some data via an IO action, and then storing that data to memory. However, to keep the the parser as pure as possible, `liftIO` has only been used where absolutely necessary. This transformer based definition makes it quite easy for parsers to interact with memory (and IO if needed). `Memory.hs` provides some actions for parsers to interact with the memory state.

```haskell
type Parser = ParsecT Void String (StateT Memory IO)
```

I also make use of Haskeline's `InputT` monad to give the REPL interactive features [7].

3 Prompt Flexibility

The parser building blocks in `Grammar.hs` and `Numbers.hs` are designed to allow for a lot of flexibility. The parser is generally not case sensitive, accepts terminating and non-terminating punctuation, and handles whitespace appropriately. Strings can be given in regular form, or can be quoted. Numbers can be given in longhand or numeric form. `parseLonghand` has been refactored to be more flexible and to fix the "thousand bug". You can also make the requests more polite with words like "please". Certain requests can also be structured in different ways. The following requests are identical to each other:

`tell me about haskell` ↔ `Please, could you tell -ME- about: "Haskell"?`

`what is twenty two minus five` ↔ `WhAt Is 22 MiNuS 5?`

`Play Bad Apple at 2x` ↔ `Play Bad Apple at two times speed`

4 Error Handling

The error handler allows custom error messages to be propagated by parsers thrown using the `fail` function. This is useful when some input is of the correct "shape" but is invalid. When the input ends abruptly, or includes a typo, the AI tries to offer help to fix the error. It does this by using a Porter2 Stemmer [10] and then calculating the Damerau-Levenshtein distance [5] to find close matches.

```
λ> How long ago was 2007-07-99
Sorry, that date is invalid.
λ> What day
I don't understand that.
The request ended abruptly, expecting any of the following words: "is", "was".
λ> What day iis it?
I don't understand that.
Did you mean "is" instead of "iis"?
```

5 Interactive REPL

The REPL is interactive, it offers line editing features, arrow key navigation, prompt history, etc. These features are implemented via the Haskeline library.

6 Skills

Each skill file exposes `parser :: Parser (IO ())` which implements some behaviour and `help :: IO ()` which gives usage information. Skills are merged into one final parser via `LGPT.Skills.mergeSkills`. By adding or omitting skills, a user can easily toggle functionality.

A help command is also automatically generated. Write `Help` to get help on every registered skill, or `Help with [SKILL]` to get help on a specific one. Though the margins of this report are too narrow to give descriptions and examples for every single command, this `Help` feature should allow a user to find out just that.

Currently, the included skills are `BrainF` [1, 3, 2], `Debug`, `Math`, `Phatic`, `Recall`, `Time`, `Video` [4, 6], and `Web` [12, 8] (citations link to online resources which I referred to when implementing these skills).

Some skills integrate quite nicely with each other. For example, `Web` can fetch remote data and store it in memory as `"that"`. `BrainF`'s `Run that` command could then evaluate the fetched data. Or `Math`'s `What is that ...` command could then perform arithmetic on it. This is demonstrated by some of the demo scripts.

6.1 Demos

Under the `demos/` folder, I have included a few shell scripts, which when run, showcases some skill's functionality. The scripts just pipe in some text to the LGPT executable.

7 Future Work

I had a good experience writing this project. However, one fundamental limitation I faced to parsing natural language like this is its rigidity. An improved version of LGPT could feature a lexing and tagging [11] process prior to parsing, possibly involving some machine learning model. Such refinement processes are typical in almost every modern Natural Language Processing pipeline. This would allow the AI to better focus on the semantics of a request, rather than its syntax, hence making it more flexible.

References

[1] Brainf. https://esolangs.org/wiki/Brainfuck.

[2] Brainf 15 puzzle. https://github.com/arkark/15puzzle-brainfuck.

[3] Brainf programs. https://www.brainfuck.org.

[4] Chafa manual. https://hpjansson.org/chafa/man/.

[5] Damerau-levenshtein distance. https://en.wikipedia.org/wiki/Damerau%E2%80%93Levenshtein_distance.

[6] Ffmpeg - the ultimate guide. https://img.ly/blog/ultimate-guide-to-ffmpeg.

[7] Going repling with haskeline. https://abhinavsarkar.net/posts/repling-with-haskeline.

[8] Lens/aeson traversals/prisms. https://www.schoolofhaskell.com/user/tel/lens-aeson-traversals-prisms.

[9] Megaparsec tutorial. https://markkarpov.com/tutorial/megaparsec.html.

[10] Stemming. https://en.wikipedia.org/wiki/Stemming.

[11] Transformation-based learning for pos tagging. https://elijahpotter.dev/articles/transformation-based-learning.

[12] wreq – http made easy for haskell. https://www.serpentine.com/software/wreq.

(All links confirmed to be accessible as of 4 April 2026)